

Vectors, Matrices, and Collision Detection

Code Documentation

Part 1:

Point vs Vector:

- In terms of math, a Point represents a specific, fixed location in space (like coordinates on a map). It has no size, no volume, and no direction, unlike a Vector...
- A Vector on the other hand represents a displacement; a length and a direction. A vector doesn't have a fixed location; pushing a vector 5 units to the right doesn't change what the vector is.

When to use them:

- Points are used when you need to store where something is, in the context of gaming maybe a point stores the spawn location of the player, or the exact spot a bullet hit a wall.
- A Vector is used when you need to describe movement or direction, again in the context of games; perhaps its used as the data structure for the velocity of a moving car, the direction an enemy is facing or gravity in general.

Q3. Dot Product Use Case: the dot product helps determine how aligned two vectors are. For example in Field of View/ Vision Cones. If you take the direction the enemy is facing and calculate the dot product against the direction from the enemy to the player, you can tell if the player is in front of/ or behind the enemy.

Q4. Cross Product Use Case: the cross product takes two vectors and returns a third vector that is perfectly perpendicular to both of them. A standard use case is finding a surface normal. If you have a triangle on a 3D terrain, calculating the cross product of two of its

edges gives us the exact direction that the surface is facing, which is required to calculate realistic lighting or figure out if the player can walk up a slope or not.

Q5. Normalization: the scaling of a vector so its magnitude (length) becomes exactly 1.0, while keeping its direction unchanged. This is important for direction vectors because if you multiply a direction by a speed variable (e.g `movementDir * 5.0f`), you need to know the direction isn't secretly adding extra speed. If you don't normalize movement vectors, a player holding "Up" and "Right" at the same time will move diagonally significantly faster than them just moving in a straight line.

Part 2:

Q7. Matrix Multiplication Order:

- The order; when transforming a 3D model from its own local space into the game's world space, the exact order of operations must be Scale, then Rotation, then Translation. Mathematically, if you are reading the matrix equation from right to left, which I did for the IExeEngine (My custom engine that uses OpenGL) it looks like $\text{WorldMatrix} = \text{Translation} \times \text{Rotation} \times \text{Scale}$.

- Order matters as matrix multiplication isn't commutative;

$$A \times B \neq B \times A$$

If operations are performed out of order, you can get completely warped results. i.e. if you Translate before you Scale, the translation distance itself gets scaled, instead of something moving 5 units it moves 10. If you Translate before Rotate, the object will rotate around the world origin (0, 0, 0) in a giant orbit, rather than spinning in place around its own centre.

Q8. Transpose & Inverse Use Cases:

- **Transpose Use Case:** a transposed matrix swaps its rows with its columns. In game development, the most common use case is dealing with Rotation Matrices. For a matrix that only contains rotation (an orthogonal matrix), its Transpose is mathematically identical to its

Inverse. Calculating a Transpose is basically free for the CPU, while calculating a true Inverse is very expensive. We use the Transpose to instantly reverse rotations, which is heavily used in shader programming (like normal mapping) to convert lighting data from World Space into Tangent Space.

- **Inverse Matrix Concept & Use Case:** an Inverse matrix completely reverses whatever transformations a matrix applies. If Matrix A moves an object 5 units up, the Inverse of A moves it 5 units down. The most common and I guess famous use case is the Camera View Matrix. In a game engine, the camera doesn't actually move through the world. Instead, you calculate the camera's world position, take the Inverse of that matrix, and apply it to the entire universe. You are literally pulling the world towards the camera.

Part 3:

Q11. Projectile Hit Detection Strategy

- **Hitscan:** for weapons that instantly hit their target the moment you hit the shoot input, you must use a Raycast (or RayTrace/ Line-Trace). You define a mathematical “Ray” starting at the gun’s barrel, then extending it infinitely (or a max range) along the gun’s forward direction vector. You can then mathematically test if this line intersects with the bounding volumes (AABBs or Capsules) of any enemies hitboxes.

- **Slow Shot (Physical projectile):** for slow-moving projectiles, you use discrete collision detection; updating a moving collision shape, like a sphere, frame-by-frame, where every frame the projectile’s position is updated based on its velocity and check for a ‘shape vs shape’ intersection.